

They propose a novel architecture to compute vectors from words.

At this time many people represented words as atomic units with no notion of similarity between words. but in this paper they hope to propose a model can learn on billions of words and also have similar words near each other, with degrees of similarity.

They build off of previous work that has shown that word vectors improve NLP applications. But they try to find a less computationally expensive model.

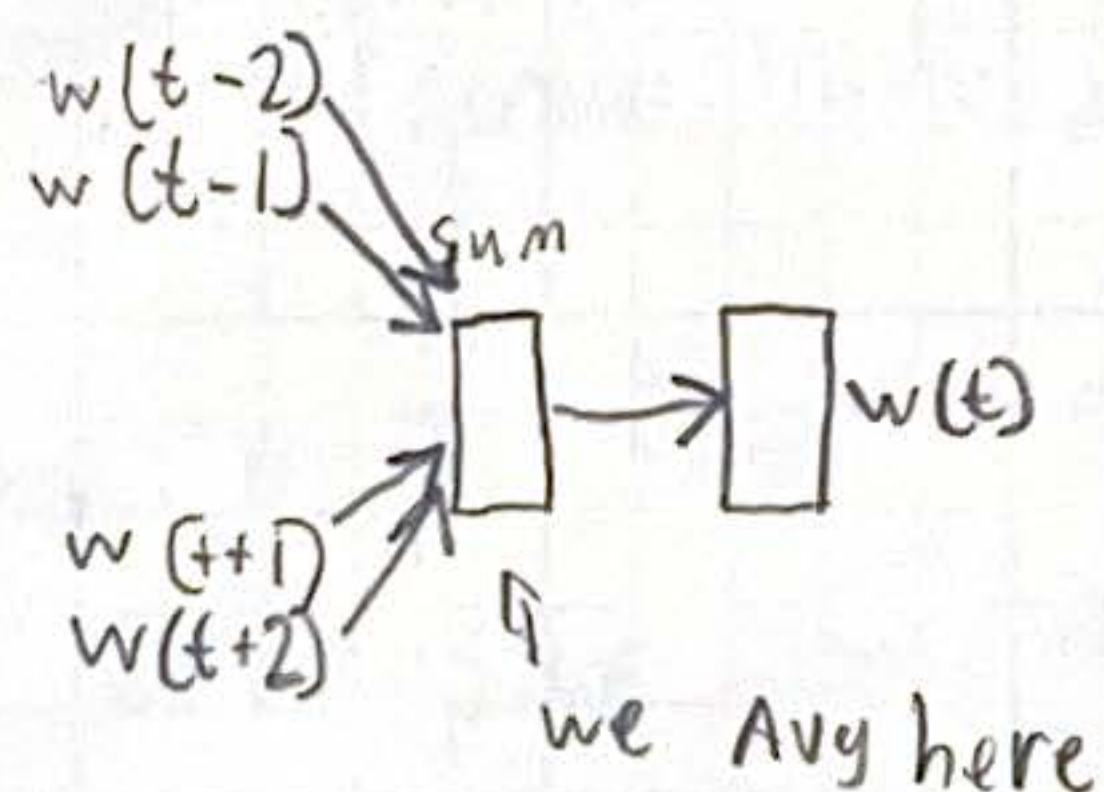
They note that the multi-hidden layers of the nn make it attractive but also result in most of the complexity so they propose to look for a simpler solution.

LET'S NOTE WHAT THE GOAL OF Models

it to have a "lookup table" (the embedded Matrix) for one-hot $\Rightarrow$ meaningful vector.

they propose 2 models for this.

- Bag of Words model



Forward during training

$\text{embeddings} = \text{embedding_matrix}[\text{words one hot}]$

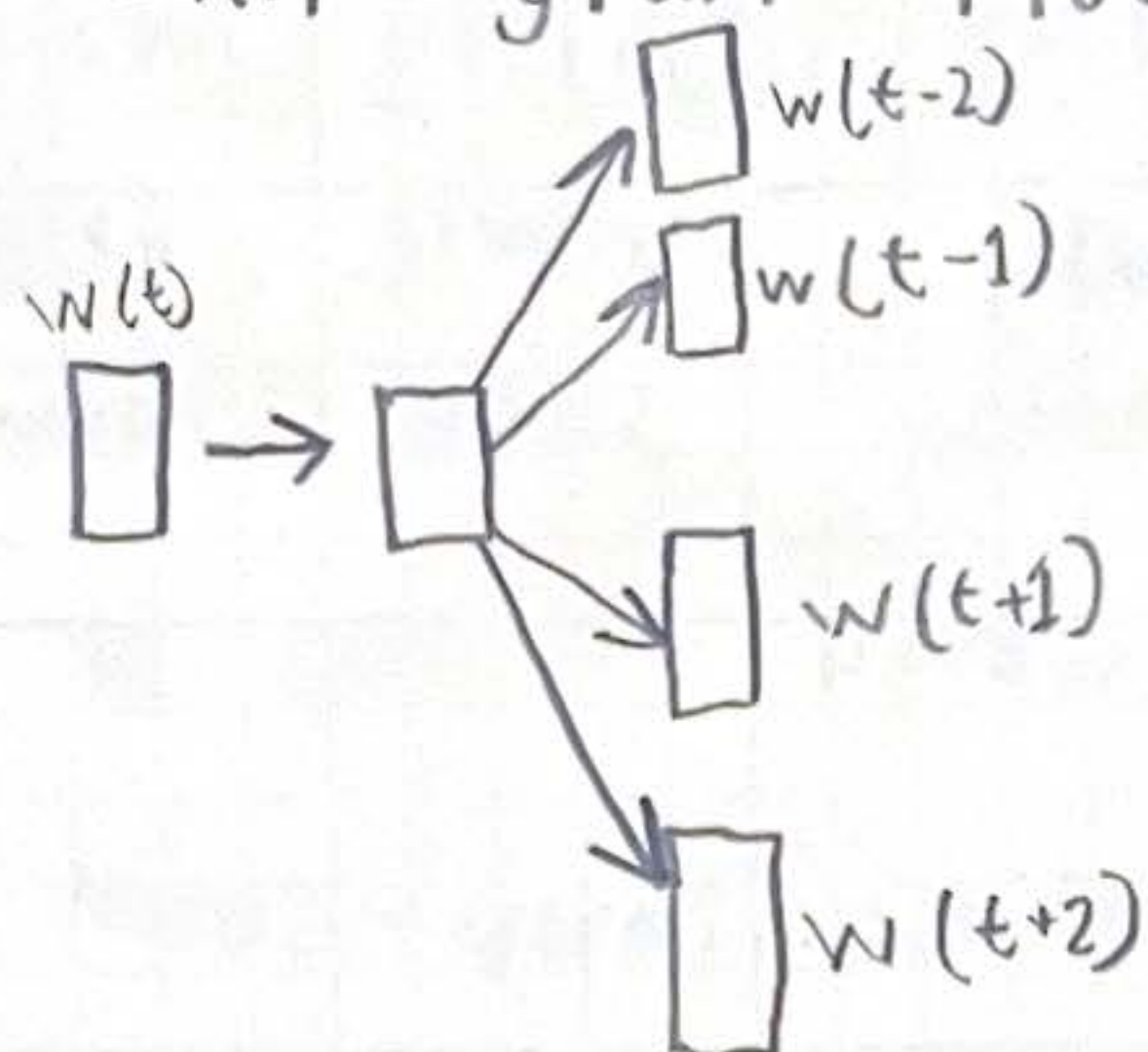
$\text{avg_embedding} = \text{sum}(\text{embedding_matrix}) / \text{len}(\text{embedding_matrix})$

$\text{logits} = \text{avg} @ \text{out_embedding.T}$

$\text{prob} = \text{soft max}(\text{logits})$

★ the Avg is more efficient but we lose some info ★

- Skip gram model training forward pass



$$\begin{aligned} \text{word embedding} &= \text{Embedding_matrix}[\text{onehot}] \\ \text{logits} &= v \text{word embedding} @ \text{out_embedding.T} \\ \text{Prob} &= \text{Softmax}(\text{logits}) \end{aligned}$$

So the goal here is to learn a good embedding matrix. If we see billions of examples +

King ^{often} → [the, ruled, kingdom]

Queen ^{often} → [royal, ruled, kingdom]

then the queen + King vectors learn to be near each other in the vector space.

Man ^{might appear near} → [person, human, individual]

Woman ^{might appear near} → [person, human, individual]

they note that Softmax across the whole thing is expensive so they do Softmax for just a few of the negative samples.

List of words

(Athens, greece) ; (oslo, Norway) Semantic

(great, greater) ; (tough, tougher) Syntactic

word1 word2 word3 word4

to evaluate performance they to the following.

$$\text{word2} - \text{word1} + \text{word3} = \text{vector-X}$$

the nearest vector to vector-X in the vector space could be word4. If not then its wrong.

they make this long list word pairs + eval the model.

The Skipgram model does Better in general particularly on Semantic word Relation.

(BOW does Better on Syntactic accuracy

Then they note significantly on the learned relationships in the model and the embedding vector arithmetic as we noted on page 01 + 02.

NOTE ON THE INTUITION:

The goal is to have meaningful embeddings but at initialization it's just a table with a bunch of Random vectors. During training the Embeddings + last layer get updated. So in (skip-gram) we pass in the one word + then get it's

Embedding * the one linear layer + that corresponds to predicted prob a word is near by. and the Predicted Prob for true near by words should be high + low for the non-near by words

03
That's how the loss is computed then
backpropagated so it updates the linear layer
& the embedding vector so that next time
it's more likely to give high prob to the
"right words".

So if King & Queen are around similar
words then the embedding vectors will be
updated to place them in similar places
Since their individual embedding vector *
the shared linear layer need to predict
similar words.